

Restful-Booker API - JIRA Test Case Format & Execution Guide

Overview

1. JIRA Test Case Template

Basic Structure in JIRA

Project: [Project Key]
Test Case ID: [Unique ID from TEST_PLAN]

2. Detailed JIRA Test Case Examples

Example 1: Happy Path Test Case

```
=== TEST CASE DETAILS ===  
Test ID: TC_PING_001  
=== PRECONDITIONS ===  
=== STEPS TO REPRODUCE ===  
=== EXPECTED RESULT ===  
=== ACTUAL RESULT ===  
=== TEST RESULT ===  
=== DEFECT REFERENCE ===  
=== NOTES & COMMENTS ===  
---
```

Example 2: Error Scenario Test Case

```
=== TEST CASE DETAILS ===  
Test ID: TC_AUTH_002  
=== PRECONDITIONS ===  
=== STEPS TO REPRODUCE ===  
=== EXPECTED RESULT ===  
Acceptable Response Example:  
Or no token field present:  
=== ACTUAL RESULT ===  
=== TEST RESULT ===  
=== NOTES ===  
---
```

Example 3: Edge Case / Boundary Test

```
=== TEST CASE DETAILS ===  
Test ID: TC_BOOKING_CREATE_011  
=== PRECONDITIONS ===  
=== STEPS TO REPRODUCE ===  
Test Data (Payload):  
=== EXPECTED RESULT ===  
Option 2 - API rejects with validation error:
```

=== ACTUAL RESULT ===

=== TEST RESULT ===

=== OBSERVATIONS ===

3. Test Execution Checklist

Before executing any test cases, verify:

- [] API server is accessible and running

4. Step-by-Step Test Execution Guide

Phase 1: PING Endpoint Testing (2 tests)

1. [TC_PING_001] Open REST client
2. [TC_PING_002] Configure timeout

Phase 2: AUTH Endpoint Testing (7 tests)

Prerequisites: Complete PING tests first

1. [TC_AUTH_001] Valid Credentials
2. [TC_AUTH_002] Invalid Username
3. [TC_AUTH_003] Invalid Password
4. [TC_AUTH_004] Missing Username
5. [TC_AUTH_005] Missing Password
6. [TC_AUTH_006] Empty Credentials
7. [TC_AUTH_007] Special Characters

Phase 3: GET Booking IDs Endpoint Testing (9 tests)

Prerequisites: Complete AUTH tests, have valid token

1. [TC_BOOKING_IDS_001] Get All Booking IDs
2. [TC_BOOKING_IDS_002] Filter by First Name
3. [TC_BOOKING_IDS_003] Filter by Last Name
4. [TC_BOOKING_IDS_004] Filter by Check-in Date
5. [TC_BOOKING_IDS_005] Filter by Check-out Date
6. [TC_BOOKING_IDS_006] Combined Filters
7. [TC_BOOKING_IDS_007] Invalid Date Format
8. [TC_BOOKING_IDS_008] No Matches
9. [TC_BOOKING_IDS_009] Case Sensitivity

Phase 4: GET Booking Endpoint Testing (8 tests)

Prerequisites: Verify at least one booking exists

1. [TC_BOOKING_GET_001] Get Booking - Happy Path
2. [TC_BOOKING_GET_002] Accept JSON Header
3. [TC_BOOKING_GET_003] Accept XML Header
4. [TC_BOOKING_GET_004] Non-Existent ID
5. [TC_BOOKING_GET_005] Invalid ID Format
6. [TC_BOOKING_GET_006] Negative ID
7. [TC_BOOKING_GET_007] Zero ID

8. [TC_BOOKING_GET_008] Very Large ID

Phase 5: CREATE Booking Endpoint Testing (15 tests)

Prerequisites: API accessible, token available

1. [TC_BOOKING_CREATE_001] Create Booking - JSON
2. [TC_BOOKING_CREATE_002] Create Booking - XML
- 3-6. [TC_BOOKING_CREATE_003 to 006] Missing Required Fields
7. [TC_BOOKING_CREATE_007] Invalid Date Format
8. [TC_BOOKING_CREATE_008] Checkout Before Checkin
9. [TC_BOOKING_CREATE_009] Negative Total Price
10. [TC_BOOKING_CREATE_010] Zero Total Price
11. [TC_BOOKING_CREATE_011] Very Long First Name
12. [TC_BOOKING_CREATE_012] Special Characters
13. [TC_BOOKING_CREATE_013] Unicode Characters
14. [TC_BOOKING_CREATE_014] Decimal Total Price
15. [TC_BOOKING_CREATE_015] Optional Field Missing

Phase 6: UPDATE Booking Endpoint Testing (8 tests)

Prerequisites: Have valid token, have existing booking to update

1. [TC_BOOKING_UPDATE_001] Update with Cookie Auth
2. [TC_BOOKING_UPDATE_002] Update with Basic Auth
3. [TC_BOOKING_UPDATE_003] Update Multiple Fields
4. [TC_BOOKING_UPDATE_004] Update Without Authentication
5. [TC_BOOKING_UPDATE_005] Update Non-Existent ID
6. [TC_BOOKING_UPDATE_006] Invalid ID Format
7. [TC_BOOKING_UPDATE_007] Update Single Field
8. [TC_BOOKING_UPDATE_008] Invalid Date Format

Phase 7: PATCH Booking Endpoint Testing (6 tests)

Prerequisites: Have valid token, have existing booking

1. [TC_BOOKING_PATCH_001] Patch First Name Only
2. [TC_BOOKING_PATCH_002] Patch Last Name Only
3. [TC_BOOKING_PATCH_003] Patch Total Price Only
4. [TC_BOOKING_PATCH_004] Patch Without Authentication
5. [TC_BOOKING_PATCH_005] Patch Non-Existent ID
6. [TC_BOOKING_PATCH_006] Patch Empty Body

Phase 8: DELETE Booking Endpoint Testing (10 tests)

Prerequisites: Have valid token, have bookings to delete

1. [TC_BOOKING_DELETE_001] Delete with Cookie Auth
2. [TC_BOOKING_DELETE_002] Delete with Basic Auth
3. [TC_BOOKING_DELETE_003] Delete Without Authentication
4. [TC_BOOKING_DELETE_004] Delete with Invalid Token
5. [TC_BOOKING_DELETE_005] Delete Non-Existent Booking
6. [TC_BOOKING_DELETE_006] Delete Invalid ID Format
7. [TC_BOOKING_DELETE_007] Verify After Deletion
8. [TC_BOOKING_DELETE_008] Delete with Expired Token

- 9. [TC_BOOKING_DELETE_009] Delete Negative ID
 - 10. [TC_BOOKING_DELETE_010] Delete Already Deleted
-

5. Test Execution Status Tracking

Use this table to track test execution:

Test ID	Test Name	Status	Pass/Fail	Notes	Date	Executed By
Status Options: Ready / In Progress / Passed / Failed / Blocked / Skipped						

6. Defect Logging Template

When a test fails, create a JIRA defect with:

- Issue Type: Bug / Defect
 - Description:
 - Affected Endpoint: [e.g., /booking]
 - Steps to Reproduce:
 - Expected Behavior:
 - Actual Behavior:
 - Environment:
 - Screenshots/Logs:
 - Workaround:
-

7. Test Metrics & Reporting

Key Metrics to Track

- Total Test Cases: 65
- Test Coverage by Endpoint:
- Overall Coverage: 100% (65/65)

Sample Test Report

- TEST EXECUTION REPORT
 - SUMMARY
 - RESULTS BY ENDPOINT
 - AUTH Endpoint:
 - [Continue for each endpoint...]
 - DEFECTS FOUND
 - [List all defects found...]
 - RECOMMENDATIONS
-

8. Tools & Resources

Recommended Tools for REST API Testing

Required Software

API Documentation Reference

9. Quality Assurance Checklist

Before marking test execution as complete:

- [] All test cases have been executed

****END OF EXECUTION GUIDE****